

SMART INVENTORY AND DECISION SUPPORT SYSTEM FOR SMALL RETAIL BUSINESSES

Technical Documentation of the Decision Support Modules

Expiry Detection · Buying Advisor · Credit Reminders · Pricing Guidance · Bilingual Interface

1. Introduction

This document explains the design and internal logic of the decision support modules of the Smart Inventory and Decision Support System. It covers what each module does, the exact rule or formula it applies, the data it depends on, and a worked example for each.

The system is a multi-tenant mobile application. The back end is built with Laravel (PHP) using a PostgreSQL database, where each registered business receives its own isolated database schema. The mobile front end is built with React Native (Expo). The modules described here implement the functional requirements FR06, FR10, FR12 and FR13 of the project proposal, together with two additional improvements: selling price guidance and a bilingual user interface.

1.1 Modules covered

Requirement	Module	Purpose
FR06	Expiry Detection and Alerts	Warn the owner about expired and soon-to-expire stock
FR12	Purchase Price History	Store the price paid for a product on every restock
FR13	Buying Advisor	Advise whether today is a good time to buy
FR10	Credit Payment Reminders	Remind the owner about overdue customer debts
Additional	Selling Price Guidance	Suggest a selling price from the buying price
Additional	Bilingual Interface	Allow the app to be used in English or Kiswahili

2. Expiry Detection and Alerts (FR06)

2.1 Problem addressed

The baseline survey established that 74% of the small retailers interviewed had unknowingly sold expired products, causing customer complaints and loss of trust. The requirement (FR06) states that the system shall automatically check expiry dates and display alerts for products that are expired or within seven days of expiry.

2.2 Data used

The module introduces no new database table. It relies on the expiry_date column that already exists in the products table, together with the stock column.

2.3 The detection rule

The check is performed on the server each time the application requests the alerts. The procedure is as follows:

1. Read the look-ahead window in days from the request. The default is 7 days, and the value is restricted to the range 1 to 90 to prevent invalid input.
2. Establish two reference points: the current date and time (now), and the threshold limit, calculated as now plus the window, taken to the end of that day.
3. Select only products that have an expiry date recorded and whose stock is greater than zero. Products no longer held in stock are excluded because no action can be taken on them.
4. Classify each selected product as EXPIRED when its expiry date is less than or equal to now.
5. Classify each selected product as EXPIRING SOON when its expiry date is later than now but less than or equal to the threshold limit.
6. Sort both groups by expiry date in ascending order, so the most urgent item appears first.
7. For each product compute a signed day difference between today and the expiry day, both reduced to the start of the day.

2.4 Interpretation of the day difference

The signed day difference gives the application a single number from which it can produce a readable label:

- A negative value means the product expired that many days ago (for example, -4 is displayed as "4 days ago").
- Zero means the product expires today.
- A positive value means the product expires in that many days (for example, +2 is displayed as "In 2 days").

2.5 Worked example

Assume today is 22 July 2026 and the window is the default 7 days. The threshold limit is therefore the end of 29 July 2026.

Product	Expiry date	Stock	Result
Fresh Milk 1L	18 July 2026	12	EXPIRED (4 days ago)
Bread loaf	24 July 2026	20	EXPIRING SOON (in 2 days)
Eggs (tray)	29 July 2026	30	EXPIRING SOON (in 7 days)
Rice 25kg	30 August 2026	8	Not reported (beyond window)
Yoghurt cup	20 July 2026	0	Not reported (no stock left)

The response therefore contains one expired product and two products expiring soon. Rice is outside the seven day window, and the yoghurt is excluded because none remains in stock.

2.6 How the alert reaches the user

The alerts are surfaced in two places so that the owner cannot miss them:

- A red banner on the Dashboard, which is the first screen shown after login. It displays the two counts, for example "2 expired · 3 expiring soon", and opens the Alerts screen when tapped. Because the Dashboard requests the alerts as soon as it loads, the warning appears within a few seconds of logging in, satisfying non-functional requirement NFR02.
- A dedicated Alerts screen, which lists the affected products in two colour-coded sections: red for expired stock and amber for stock expiring soon. Each entry shows the product name, its expiry date, how many days it is overdue or remaining, and the quantity still held.

2.7 Implementation location

Layer	File
Back end (logic)	backend/app/Http/Controllers/ExpiryAlertController.php
Back end (route)	GET /api/products/expiry-alerts?days=7
Front end (screen)	frontend/screens/Alerts/ExpiryAlerts.jsx
Front end (banner)	frontend/screens/Dashboard/Dashboard.jsx

3. Purchase Price History and Buying Advisor (FR12 and FR13)

3.1 Problem addressed

Retailers frequently purchase stock when prices are high because they have no record of what they previously paid and therefore no basis for comparison. Requirement FR12 states that the system shall store historical purchase prices for each product, and FR13 states that the system shall recommend restocking when the current market price is lower than the average of the last three purchase prices.

3.2 Storing the price history (FR12)

A new table, `purchase_prices`, was added to the tenant schema. Each row records one purchase of one product:

Column	Type	Meaning
<code>product_id</code>	foreign key	The product bought; deletes with the product
<code>price</code>	decimal(15,2)	Price paid for one unit on this purchase
<code>currency</code>	string(5)	Currency of the purchase
<code>quantity</code>	decimal(10,2)	How much was bought; added to stock
<code>note</code>	string, nullable	Optional remark, such as the supplier name
<code>purchased_at</code>	datetime	When the purchase was made

Indexes are placed on `product_id` and `purchased_at` so that the most recent purchases of a product can be retrieved quickly.

Recording a purchase performs two actions in a single step, which keeps the workflow natural for the shopkeeper. First, the price paid is appended to the product history. Second, the quantity bought is added to the current stock, and the product's current buying price is updated to the newly paid price. If the product does not permit fractional quantities, the quantity added is reduced to a whole number.

3.3 The recommendation rule (FR13)

The decision is computed on the server so that the rule is testable and identical on every device. Given a product and a market price quoted today, the procedure is:

1. Retrieve the three most recent purchases of that product, ordered by purchase date descending.
2. If fewer than three purchases exist, return the status `INSUFFICIENT_DATA` and give no advice. The system deliberately does not guess from one or two records.
3. Otherwise compute the benchmark as the arithmetic mean of those three prices.
4. Compare the quoted market price with the benchmark. If the market price is strictly lower than the benchmark, the recommendation is `BUY`. Otherwise the recommendation is `WAIT`.
5. Compute the saving as the benchmark minus the market price, and express it as a percentage of the benchmark.

3.4 The formulas

```
benchmark = (P1 + P2 + P3) / 3      where P1..P3 are the last three purchase prices
decision  = BUY  if market_price < benchmark
           WAIT if market_price >= benchmark
saving    = benchmark - market_price
saving %  = (saving / benchmark) x 100
```

3.5 Worked example

A shop has recorded three purchases of rice: 5,000 TZS, 5,200 TZS and 4,800 TZS per kilogram.

$$\text{benchmark} = (5,000 + 5,200 + 4,800) / 3 = 15,000 / 3 = 5,000 \text{ TZS}$$

Two different quotations are then tested against this benchmark:

Market price today	Benchmark	Decision	Explanation
4,500 TZS	5,000 TZS	BUY	4,500 < 5,000; saving 500, i.e. 10% below average
5,300 TZS	5,000 TZS	WAIT	5,300 is above the average; no saving
5,000 TZS	5,000 TZS	WAIT	Equal to the average, so there is no advantage

3.6 Design decisions and their justification

- Why the average of the last three purchases: this is the rule stated in FR13. Three observations smooth out a single unusual price while remaining recent enough to reflect the present market, unlike an average over the whole history.
- Why a strict comparison: when the market price equals the benchmark there is no saving, so the system advises WAIT rather than BUY.
- Why a minimum of three records: with only one or two purchases the average is not a meaningful benchmark. Reporting INSUFFICIENT_DATA is more honest than issuing weak advice, and the interface tells the user how many further purchases are required.
- Why the decision is computed on the server: the rule is a single piece of business logic that can be unit tested independently of the interface, and it cannot be altered by the client.

3.7 How the user interacts with it

- On each product in the Inventory list there is a Record Purchase action. It captures the price paid, the quantity bought, the date, and an optional supplier note, and it simultaneously increases the stock.
- The Buying Advisor screen lists every product with its benchmark (the average of the last three purchases) and the price last paid. The user types the price being quoted today and receives an immediate BUY or WAIT answer, together with the percentage saving where relevant.

3.8 Implementation location

Layer	File or endpoint
Database	backend/database/migrations/tenant/..._create_purchase_prices_table.php
Model	backend/app/Models/PurchasePrice.php
Logic	backend/app/Http/Controllers/PurchasePriceController.php
Record a purchase	POST /api/products/{product}/purchases
Price history	GET /api/products/{product}/purchases
Benchmarks	GET /api/purchase-recommendations
Decision engine	POST /api/purchase-check
Front end	frontend/screens/BuyingAdvisor/BuyingAdvisor.jsx
Front end	frontend/screens/Inventory/Modal/RecordPurchaseModal.jsx

4. Credit Payment Reminders (FR10)

4.1 Problem addressed

The survey found that 62% of retailers had lost money because customers with outstanding debts were forgotten or the records were lost. FR10 requires the system to notify the owner of credit payments that are overdue.

4.2 The rule

The module examines credit sales only. A transaction qualifies for consideration when its payment method is CREDIT, its status is not COMPLETED, and a due date has been recorded. Two groups are then produced:

- OVERDUE: the due date has already passed.
- DUE SOON: the due date falls between now and the end of the look-ahead window, which defaults to 3 days.

Both groups are sorted by due date ascending, so the most overdue debt appears first. As in the expiry module, each entry carries a signed day difference, so the interface can state "Overdue by 5 days" rather than only showing a date.

4.3 How the reminder reaches the user

An orange banner appears on the Dashboard immediately after login, showing the counts, for example "2 overdue · 1 due soon". Tapping it opens the Creditors screen, where each late debt is labelled with the number of days by which it has been exceeded.

4.4 Note on the form of the reminder

The proposal describes this requirement as a local push notification. In the present implementation the reminder is delivered inside the application: it is shown as soon as the owner opens the app, which is the point at which the information is acted upon. A true operating-system push notification, which would appear while the application is closed, would require an additional notification library and device permissions. This distinction is stated here so that the implemented behaviour is described accurately.

4.5 Implementation location

Layer	File or endpoint
Logic	backend/app/Http/Controllers/CreditReminderController.php
Route	GET /api/credit-reminders?days=3
Front end (banner)	frontend/screens/Dashboard/Dashboard.jsx
Front end (list)	frontend/screens/Creditors/CreditorsList.jsx

5. Selling Price Guidance

The problem statement of the proposal notes that most retailers set prices arbitrarily, without considering the purchase cost or a desired profit margin. To address this, the Add Product and Edit Product forms now provide pricing guidance.

As soon as a buying price is entered, the form offers three suggested selling prices, calculated at margins of 20%, 30% and 50%:

```
suggested selling price = round( buying price x (1 + margin / 100) )
```

For a buying price of 5,000 TZS the form therefore offers 6,000 TZS at 20%, 6,500 TZS at 30% and 7,500 TZS at 50%. Selecting one of the suggestions fills in the selling price field. The user remains free to type any other price; the suggestions are guidance and not a restriction.

6. Bilingual Interface (English and Kiswahili)

The intended users are small retail traders, many of whom are more comfortable reading Kiswahili than English. The application therefore supports both languages, and the user may switch between them at any time.

6.1 How it works

6. A dictionary file holds every interface phrase under a short key, with one English value and one Kiswahili value for each key.
7. A language provider supplies a translation function to every screen. Given a key, it returns the phrase in the language currently selected.
8. If a key is missing from the selected language, the function falls back to the English value, and finally to the key itself. This guarantees that no screen can ever display an empty label.
9. The translation function also substitutes values into a phrase, so that counts and names can be placed correctly within a sentence in either language.
10. The chosen language is saved on the device, so the application starts in that language on the next launch.

6.2 Coverage

The following screens are available in both languages: the navigation menu, Login, Create Workspace, Create Account, Dashboard, Inventory, Add and Edit Product, Record Purchase, Categories with add and edit, Expiry Alerts, Buying Advisor, Creditors, Transactions and Report. The advice text of the Buying Advisor is also produced in the selected language.

7. Corrections Made During Testing

The following defects were identified during testing of the application and corrected.

Area	Problem observed	Correction applied
Saving a product	The product was not saved and no confirmation appeared	The form sent the data in the file-upload format even when no image was attached, which the browser could not encode correctly. It now sends ordinary structured data unless an image is actually being uploaded.
Expiry date field	The date picker did not respond	The date component used has no web implementation. A standard date field is now used on the web, and the device picker is retained on the phone, with a calendar view and a Done button.
Typing in forms	Fields would not accept text	A dismiss-keyboard wrapper was removing focus from a field as soon as it was selected. The wrapper is now applied only on mobile devices, where it is required.
Product category	A product could not be saved without a category	The category is now optional, which matches the database design where the field already permitted no value.

8. Summary of Application Programming Interfaces Added

Method	Endpoint	Purpose
GET	/api/products/expiry-alerts	Expired and soon-to-expire stock
POST	/api/products/{product}/purchases	Record a purchase and increase stock
GET	/api/products/{product}/purchases	Purchase price history of a product
GET	/api/purchase-recommendations	Benchmark figures for every product
POST	/api/purchase-check	Return BUY or WAIT for a quoted price
GET	/api/credit-reminders	Overdue and soon-due credit sales

All of these endpoints require an authenticated user and operate strictly within the schema of the business to which that user belongs.

9. Verification Performed

- The database schema was created and the tenant migrations executed successfully, confirming that the purchase_prices table is created for every newly registered business.
- Each new endpoint was called with a valid authentication token and returned the expected successful response.
- A product was created through the interface and stored correctly, including its expiry date.
- The buying recommendation was checked against the worked example in section 3.5, returning BUY at 4,500 TZS with a 10% saving and WAIT at 5,300 TZS.
- The interface was displayed in both English and Kiswahili, and the selected language persisted after the application was restarted.

10. Anticipated Examination Questions

The following questions are likely to arise in relation to these modules. Concise answers are provided for reference.

Why does the Buying Advisor use three purchases rather than all of them?

Requirement FR13 specifies the last three purchases. Three observations are enough to reduce the effect of a single unusual price, while remaining recent enough to represent the current market. An average over the entire history would react too slowly to price movements.

What happens if a product has only one or two recorded purchases?

The system returns the status `INSUFFICIENT_DATA` and gives no recommendation, and the screen states how many further purchases are needed. Offering advice from one or two records would not be defensible.

Why are products with zero stock excluded from the expiry alerts?

An alert is intended to prompt an action, such as discounting or removing goods. If none of the product remains, no action is possible and the alert would merely be noise.

Where is the decision logic located, and why?

In the Laravel controllers on the server. This keeps the rule in one place, allows it to be tested independently of the interface, ensures every device receives the same answer, and prevents the rule from being altered on the client.

How does the system keep the data of different businesses separate?

Each registered business is allocated its own PostgreSQL schema. When a request arrives, the tenant is identified from a request header and the database connection is switched to that schema, so queries can only reach the data of that business.

Is the credit reminder a true push notification?

No. It is an in-application reminder shown on the Dashboard as soon as the owner opens the application. This is stated openly in section 4.4, together with what a full push notification would additionally require.

End of document